41039 Programming 1

Revision of Week 6: building reliable classes



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Today's revision: 10 questions, about 35 minutes

#	Type	Topic
Q1	Fill in the blanks	Contracts, `private`, references
Q2	Multiple choice	Which `take(n)` keeps the contract?
Q3	Multiple choice	Which line does not compile?
Q4	Predict the output	Ticket numbers (Java)
Q5	Predict the output	A shelf has a book (Python)
Q6	Find the bug	The car park (Java, 4 bugs)
Q7	Find the bug	The team (Python, 3 bugs)
Q8	Write the code	A counter with a ceiling (Java)
Q9	Write the code	A playlist of its own (Python)
Q10	Write the code	Order numbers (Java)

How it works

1. Read the question and answer it **alone** first. Write your answer down.
2. Then compare with a neighbour.
3. The answer appears on the **next click**.

Week 6 in five ideas (read this first)

1. **Contract.** An *invariant* is a rule that is true for every object, e.g. $0 \leq$ `fuel` $\leq$ `capacity`. Each method **checks before it changes**, so a refused call returns `false` and changes nothing.
2. **`private` vs `_`.** Java `private`: the compiler refuses access from outside the class. Python `self._x`: only a *convention* ("please do not touch").
3. **References.** `new` makes an object. `b = a` copies the *reference*, not the object: both names now reach the same object.
4. **Shared vs per-object data.** Java `static` field / Python class attribute: **one** copy for the whole class. Instance field / `self.x = ...` in `__init__`: one copy **per object**.
5. **Composition.** An object can *have* another object as a field: a `Shelf` has a `Book`. Two holders may share one object.

```
class Ticket {
    static int issued;  // one, shared
    final int number;   // one per ticket
    private Seat seat;  // has a Seat
}
```

Q1 Fill in the blanks Contracts and references

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- A rule that must hold for every object, before and after each public method, is an **(1)**.
- A method that only reports state is a **(2)**. One that changes state is a **(3)**.
- Java's **(4)** makes the compiler refuse any use of a field from outside its class. Python's leading `_` is only a **(5)**.
- `b = a` copies a **(6)**, so both names reach one object.

Word bank (2 min, alone)

a) invariant	**f)** getter
b) mutator	**g)** static
c) convention	**h)** object
d) reference	**i)** public
e) private	**j)** constructor

Q1 Fill in the blanks Contracts and references

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- A rule that must hold for every object, before and after each public method, is an **(1)**.
- A method that only reports state is a **(2)**. One that changes state is a **(3)**.
- Java's **(4)** makes the compiler refuse any use of a field from outside its class. Python's leading `_` is only a **(5)**.
- `b = a` copies a **(6)**, so both names reach one object.

Answers

(1) a) invariant
(2) f) getter **(3)** b) mutator
(4) e) private **(5)** c) convention
(6) d) reference
Not used: g) static, h) object, i) public, j) constructor.

The constructor must **establish** the invariant. Every public method must **keep** it.

Q2 Multiple choice — Which `take(n)` keeps the contract?

Contract: accept only $0 < n \leq$ `stock`; a refused request returns `false` and changes **nothing**. `stock` starts at 4.

Version 1

```
boolean take(int n) {
    if (n <= 0 || n > stock)
        return false;
    stock -= n;
    return true;
}
```

Version 2

```
boolean take(int n) {
    stock -= n;
    if (stock < 0)
        return false;
    return true;
}
```

Version 3

```
boolean take(int n) {
    if (n > stock)
        return false;
    stock -= n;
    return true;
}
```

Choose one (90 seconds)

A. Version 1 only
B. Versions 1 and 3
C. Versions 2 and 3
D. All three

Hint: all three compile. Try `take(9)` (too many), then `take(-5)` (negative) on each version, and watch `stock`.

Q2 Multiple choice Which `take(n)` keeps the contract?

Contract: accept only $0 < n \leq$ `stock`; a refused request returns `false` and changes **nothing**. `stock` starts at 4.

Version 1

```
boolean take(int n) {
    if (n <= 0 || n > stock)
        return false;
    stock -= n;
    return true;
}
```

Version 2

```
boolean take(int n) {
    stock -= n;
    if (stock < 0)
        return false;
    return true;
}
```

Version 3

```
boolean take(int n) {
    if (n > stock)
        return false;
    stock -= n;
    return true;
}
```

Choose one (90 seconds)

✓ **A.** Version 1 only

B. Versions 1 and 3

C. Versions 2 and 3

D. All three

Answer: A

V1 ✓ `take(9)`, `take(-5)`: both `false`, stock stays 4.

V2 ✗ changes *before* it checks: `take(9)` says `false` but stock is now -5.

V3 ✗ forgets $n > 0$: `take(-5)` says `true` and stock *grows* to 9.

Q3 Multiple choice Which line does not compile?

```
class Account {
    private int balance = 0;
    static int created = 0;
    Account() { created++; }
    void deposit(int n) {
        if (n > 0) balance += n;
    }
    int getBalance() { return balance; }
}
```

Choose one (60 seconds)

A. `a.deposit(50);`

B. `Account.created = 7;`

C. `a.balance = 100;`

D. `int b = a.getBalance();`

`main` is in a **different** class, `Demo`, in the same file, and starts with:

```
Account a = new Account();
```

Each option is the next line of `main`. Exactly **one** of them does not compile.

Q3 Multiple choice Which line does not compile?

```
class Account {
    private int balance = 0;
    static int created = 0;
    Account() { created++; }
    void deposit(int n) {
        if (n > 0) balance += n;
    }
    int getBalance() { return balance; }
}
```

Choose one (60 seconds)

A. `a.deposit(50);`

B. `Account.created = 7;`

✓ **C.** `a.balance = 100;`

D. `int b = a.getBalance();`

`main` is in a **different** class, `Demo`, in the same file, and starts with:

```
Account a = new Account();
```

Each option is the next line of `main`. Exactly **one** of them does not compile.

Answer: C

`balance` is `private`: only code inside `Account` may use it. The compiler says: `balance has private access in Account`

A, D compile: methods are the normal way in.

B compiles too, because `created` is not `private`. `static` means "shared", not "hidden".

Q4 Predict the output Ticket numbers

```
class Ticket {
    static int issued = 0;
    final int number;
    Ticket() {
        issued++;
        number = issued;
    }
}
// in main, in another class Demo
Ticket a = new Ticket();
Ticket b = new Ticket();
Ticket.issued = 10;
Ticket c = new Ticket();
Ticket d = b;
System.out.println(a.number + " " + b.number);
System.out.println(c.number + " " + d.number);
System.out.println(Ticket.issued);
```

Predict (90 seconds, alone)

1. Write down the **three** lines that are printed.

2. Which line lets other code break the numbering? How do you stop it?

Q4 Predict the output Ticket numbers

```
class Ticket {
    static int issued = 0;
    final int number;
    Ticket() {
        issued++;
        number = issued;
    }
}
// in main, in another class Demo
Ticket a = new Ticket();
Ticket b = new Ticket();
Ticket.issued = 10;
Ticket c = new Ticket();
Ticket d = b;
System.out.println(a.number + " " + b.number);
System.out.println(c.number + " " + d.number);
System.out.println(Ticket.issued);
```

Answers

1. `1 2`
`11 2`
`11`

`issued` is `static`: **one** counter for all tickets. `number` is per ticket and set once, so `a`, `b` keep 1 and 2. `Ticket.issued = 10` moved the shared counter, so `c`'s constructor makes it 11. `d = b` makes no ticket: `d` *is* `b`.

2. `Ticket.issued = 10;` jumps from 2 to 10. Make `issued` `private`: then only code inside `Ticket` can change it.

Q5 Predict the output A shelf has a book

```
class Book:
    def __init__(self, title):
        self.title = title

class Shelf:
    def __init__(self, book):
        self.book = book
    def __str__(self):
        return "Shelf with " + self.book.title

b = Book("Dune")
s1 = Shelf(b)
s2 = Shelf(b)
s2.book.title = "Emma"
s3 = Shelf(Book("Dune"))
print(s1)
print(s3)
print(s1.book is s2.book, s1.book is s3.book)
```

`x is y` is `True` only when `x` and `y` are the **same object**.

Predict (90 seconds, alone)

1. Write down the **three** lines that are printed.

2. Change the line `s2 = Shelf(b)` so that `s2` gets its **own** book.

Q5 Predict the output A shelf has a book

```
class Book:
    def __init__(self, title):
        self.title = title

class Shelf:
    def __init__(self, book):
        self.book = book
    def __str__(self):
        return "Shelf with " + self.book.title

b = Book("Dune")
s1 = Shelf(b)
s2 = Shelf(b)
s2.book.title = "Emma"
s3 = Shelf(Book("Dune"))
print(s1)
print(s3)
print(s1.book is s2.book, s1.book is s3.book)
```

`x is y` is `True` only when `x` and `y` are the **same object**.

Answers

1.
```
Shelf with Emma
Shelf with Dune
True False
```
A shelf **has a** book. `s1` and `s2` hold **the same** `Book`, so renaming it through `s2` shows through `s1`. `s3`'s book is a different object with the same title.

2. `s2 = Shelf(Book(b.title))`: a **new** `Book` object.

Q6 Find the bug The car park (Java)

```java
public class CarPark {
    private int capacity;
    private static int parked;
    public CarPark(int cap) { capacity = cap; }
    public boolean park() {
        if (parked > capacity) return false;
        parked++;
        return true;
    }
    public boolean leave() {
        parked--;
        return true;
    }
    public static int free() {
        return capacity - parked;
    }
}
```

✗ **Find 4 bugs** (3 min): 1 compile error, 3 logic errors. Give each line number and its fix.

What the class should do

- `park()` returns `false` when full
- `leave()` returns `false` when empty
- `free()` gives the number of free spaces
- Two car parks never share their counts

Test

```java
CarPark a = new CarPark(2);
CarPark b = new CarPark(5);
System.out.println(a.park() + " "
    + a.park() + " " + a.park());
System.out.println(b.free());
System.out.println(b.leave() + " "
    + b.leave() + " " + b.free());
```

Expected output

```
true true false
5
false false 5
```

Q6 Find the bug The car park (Java)

```
1  public class CarPark {
2      private int capacity;
3      private static int parked;
4      public CarPark(int cap) { capacity = cap; }
5      public boolean park() {
6          if (parked > capacity) return false;
7          parked++;
8          return true;
9      }
10     public boolean leave() {
11         parked--;
12         return true;
13     }
14     public static int free() {
15         return capacity - parked;
16     }
17 }
```

✗ **Find 4 bugs** (3 min): 1 compile error, 3 logic errors. Give each line number and its fix.

✓ The 4 fixes

- **Line 14** (compile error, reported at line 15): a `static` method has no object, so it cannot read `capacity`. Remove `static`.
- **Line 3**: `static` gives **all** car parks one shared count. Remove `static`.
- **Line 6**: `>` lets a 3rd car into 2 spaces. Use `>=`.
- **Line 11**: no check, so `parked` can go negative. Add `if (parked == 0) return false;` first.

Fix only line 14 and a still takes 3 cars.

Q7 Find the bug The team (Python)

```
1  class Team:
2      members = []
3      def __init__(self, name, size):
4          self.name = name
5          self.size = size
6      def join(self, person):
7          self.members.append(person)
8          if len(self.members) > self.size:
9              return False
10         return True
11     def leave(self, person):
12         self.members.remove(person)
13         return True
14
15 red = Team("red", 2)
16 blue = Team("blue", 3)
17 print(red.join("An"), red.join("Binh"),
18       red.join("Chi"))
19 print(red.members, blue.members)
20 print(red.leave("Dung"))
```

✗ **Find 3 bugs** in lines 1–13 (3 min).

Give each line number and its fix.

What the class should do

- Each team has its **own** list of members
- `join`: if the team is full, return `False` and change nothing
- `leave`: if the person is not in the team, return `False`

Expected output

```
True True False
['An', 'Binh'] []
False
```

✗ Actual output (crashes at line 12)

```
True True False
['An', 'Binh', 'Chi'] ['An', 'Binh', 'Chi']
ValueError: list.remove(x): x not in list
```

Q7 Find the bug The team (Python)

```
1  class Team:
2      members = []
3      def __init__(self, name, size):
4          self.name = name
5          self.size = size
6      def join(self, person):
7          self.members.append(person)
8          if len(self.members) > self.size:
9              return False
10         return True
11     def leave(self, person):
12         self.members.remove(person)
13         return True
14
15 red = Team("red", 2)
16 blue = Team("blue", 3)
17 print(red.join("An"), red.join("Binh"),
18       red.join("Chi"))
19 print(red.members, blue.members)
20 print(red.leave("Dung"))
```

✗ Find 3 bugs in lines 1–13 (3 min).
Give each line number and its fix.

✓ The 3 fixes

- **Line 2**: a list in the class body is **one** list shared by every team, so `blue` sees `red`'s members. Delete it; write `self.members = []` in `__init__`.
- **Lines 7–8**: adds *before* it checks, so the refused `"Chi"` stays in. Check first, with `>=`: `if len(self.members) >= self.size: return False`, then append.
- **Line 12**: `remove` crashes when the person is absent. First: `if person not in self.members: return False`.

Q8 Write the code A counter with a ceiling (Java)

Finish the `TODO`s (alone, 4 min). `value` starts at 0 and must **never** go above `max`.

- `increment()`: at `max`, return `false` and change nothing; otherwise add 1 and return `true`.
- `reset()`: set `value` back to 0.

Skeleton

```
class BoundedCounter {
    private final int max;
    private int value = 0;
    BoundedCounter(int max) {
        this.max = max;
    }
    boolean increment() {
        // TODO
        return false;
    }
    void reset() { /* TODO */ }
    int getValue() { return value; }
}
```

Test (in `main`)

```
BoundedCounter c = new BoundedCounter(2);
System.out.println(c.increment() + " "
    + c.increment() + " " + c.increment());
System.out.println(c.getValue());
c.reset();
System.out.println(c.getValue());
```

Must print

```
true true false
2
0
```

Q8 Write the code A counter with a ceiling (Java)

Finish the `TODO`s (alone, 4 min). `value` starts at 0 and must **never** go above `max`.

- `increment()`: at `max`, return `false` and change nothing; otherwise add 1 and return `true`.
- `reset()`: set `value` back to 0.

Skeleton

```
class BoundedCounter {
    private final int max;
    private int value = 0;
    BoundedCounter(int max) {
        this.max = max;
    }
    boolean increment() {
        // TODO
        return false;
    }
    void reset() { /* TODO */ }
    int getValue() { return value; }
}
```

✓ One answer

```
class BoundedCounter {
    private final int max;
    private int value = 0;
    BoundedCounter(int max) {
        this.max = max;
    }
    boolean increment() {
        if (value >= max) return false;
        value++;
        return true;
    }
    void reset() { value = 0; }
    int getValue() { return value; }
}
```

Q9 Write the code A playlist of its own (Python)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `__init__`: give **each** playlist its own empty list (not a class attribute: see Q7).
- `add(song)`: return `False` if `song` is `""` or already in the list; otherwise add it and return `True`.
- `__str__`: `"<n> songs: <names>"`, e.g. `"2 songs: A, B"`; if empty, just `"0 songs"`.

Skeleton

```
class Playlist:
    def __init__(self):
        pass  # TODO
    def add(self, song):
        # TODO
        return False
    def __str__(self):
        # TODO
        return "0 songs"
```

Test

```
a = Playlist()
b = Playlist()
print(a.add("Hello"), a.add("Hello"))
print(a.add(""))
a.add("Yesterday")
print(a)
print(b)
```

Must print

```
True False
False
2 songs: Hello, Yesterday
0 songs
```

Q9 Write the code A playlist of its own (Python)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `__init__`: give **each** playlist its own empty list (not a class attribute: see Q7).
- `add(song)`: return `False` if `song` is `""` or already in the list; otherwise add it and return `True`.
- `__str__`: `"<n> songs: <names>"`, e.g. `"2 songs: A, B"`; if empty, just `"0 songs"`.

Skeleton

```
class Playlist:
    def __init__(self):
        pass  # TODO
    def add(self, song):
        # TODO
        return False
    def __str__(self):
        # TODO
        return "0 songs"
```

✓ One answer

```
class Playlist:
    def __init__(self):
        self._songs = []
    def add(self, song):
        if song == "" or song in self._songs:
            return False
        self._songs.append(song)
        return True
    def __str__(self):
        text = str(len(self._songs)) + " songs"
        if self._songs:
            text += ": " + ", ".join(self._songs)
        return text
```

Q10 Write the code Order numbers (Java)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- One counter **shared** by all orders, starting at 1. No other class may change it.
- Constructor: give this order the counter's current value, then add 1 to the counter.
- `toString()`: return `"Order #<number>: <item>"` (`println(a)` calls it for you).

Skeleton

```
class Order {
    // TODO: the shared counter
    private final int number;
    private final String item;
    Order(String item) {
        this.item = item;
        this.number = 0;  // TODO
    }
    public String toString() {
        return "";  // TODO
    }
}
```

Test (in `main`)

```
Order a = new Order("tea");
Order b = new Order("coffee");
System.out.println(a);
System.out.println(b);
System.out.println(new Order("cake"));
```

Must print

```
Order #1: tea
Order #2: coffee
Order #3: cake
```

Q10 Write the code Order numbers (Java)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- One counter **shared** by all orders, starting at 1. No other class may change it.
- Constructor: give this order the counter's current value, then add 1 to the counter.
- `toString()`: return `"Order #<number>: <item>"` (`println(a)` calls it for you).

Skeleton

```
class Order {
    // TODO: the shared counter
    private final int number;
    private final String item;
    Order(String item) {
        this.item = item;
        this.number = 0;  // TODO
    }
    public String toString() {
        return "";  // TODO
    }
}
```

✓ One answer

```
class Order {
    private static int nextNumber = 1;
    private final int number;
    private final String item;
    Order(String item) {
        this.item = item;
        this.number = nextNumber;
        nextNumber++;
    }
    public String toString() {
        return "Order #" + number + ": " + item;
    }
}
```

Looking ahead: when a shelf needs more than three books

Last week: one field per book

```
class Shelf {
    private Book first, second, third;
    boolean add(Book b) {
        if (first == null) first = b;
        else if (second == null) second = b;
        else if (third == null) third = b;
        else return false;
        return true;
    }
}
```

Think (1 minute)

The library wants shelves of **40** books, and to take a book out of the **middle**. What must change?

Looking ahead: when a shelf needs more than three books

Last week: one field per book

```
class Shelf {
    private Book first, second, third;
    boolean add(Book b) {
        if (first == null) first = b;
        else if (second == null) second = b;
        else if (third == null) third = b;
        else return false;
        return true;
    }
}
```

Think (1 minute)

The library wants shelves of **40** books, and to take a book out of the **middle**. What must change?

40 fields and 40 `if`s, and taking out a middle book leaves a **hole** to manage by hand. Instead, keep the books in a list that grows:

```
class Shelf {
    private final ArrayList<Book> books
        = new ArrayList<>();
    boolean add(Book b) {
        books.add(b);
        return true;
    }
    int size() { return books.size(); }
}
```

Python: `self.books = []`. Today: **lists** in Java and Python, and the type in <Book>: **generics**.

You should now be able to...

Contracts and references

- ✓ state a class's **invariant** and each method's contract (Q1, Q2)
- ✓ check **before** you change, so a refusal changes nothing (Q2, Q6, Q7, Q8)
- ✓ use `private` (Java) and `_` (Python) for what they are (Q1, Q3)
- ✓ trace names and objects separately: `b = a` copies a reference (Q4, Q5)

Shared data and composition

- ✓ choose shared (`static`, class attribute) or per-object data (Q4, Q6, Q7, Q10)
- ✓ give each Python object its own list in `__init__` (Q7, Q9)
- ✓ build a class that **has** another object (Q5)
- ✓ write `toString` / `__str__` that returns text (Q9, Q10)

Next: Week 7 – Lists and generics

`ArrayList` and Python lists, removing safely while you loop, linked nodes and queues, and type parameters.